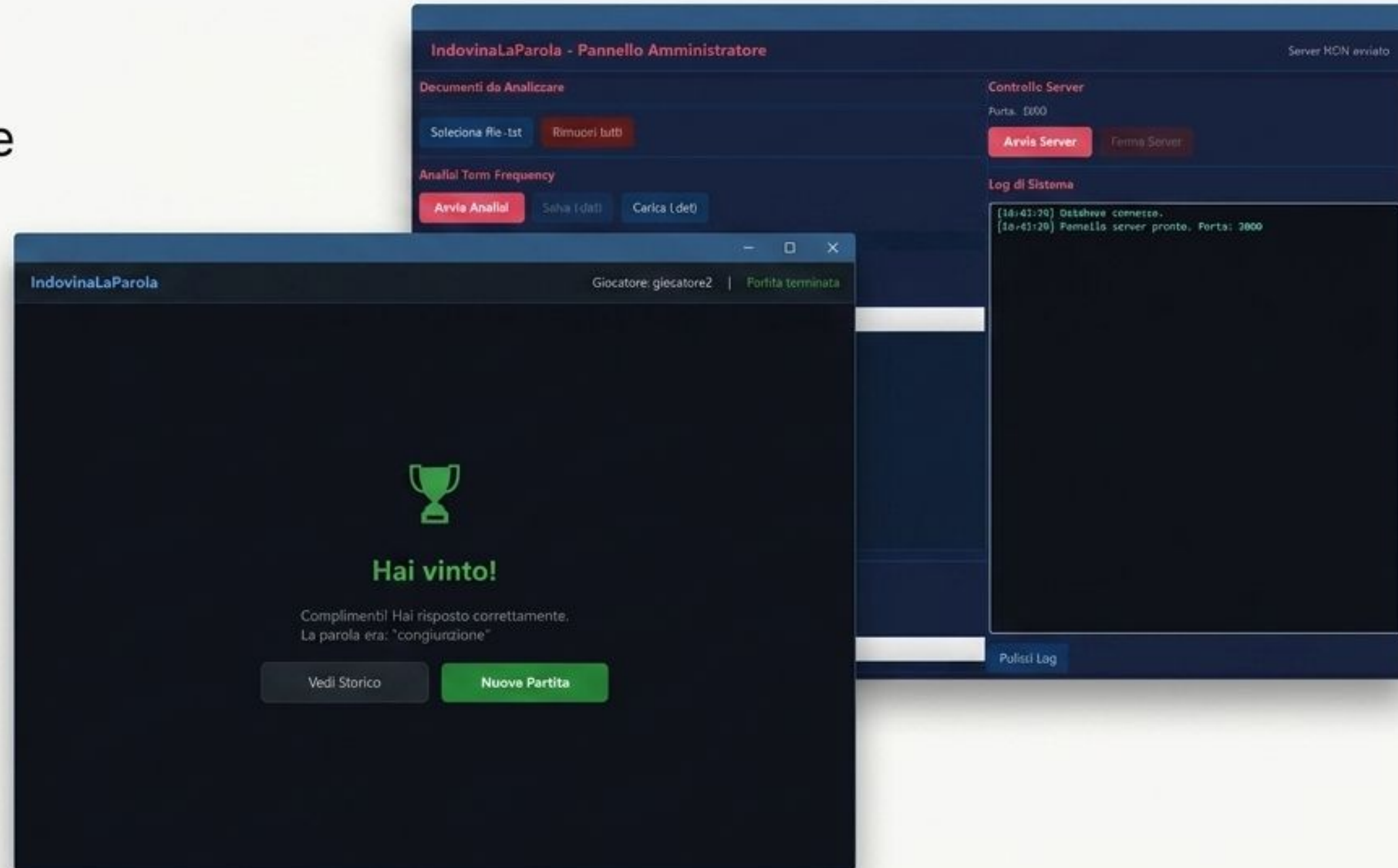


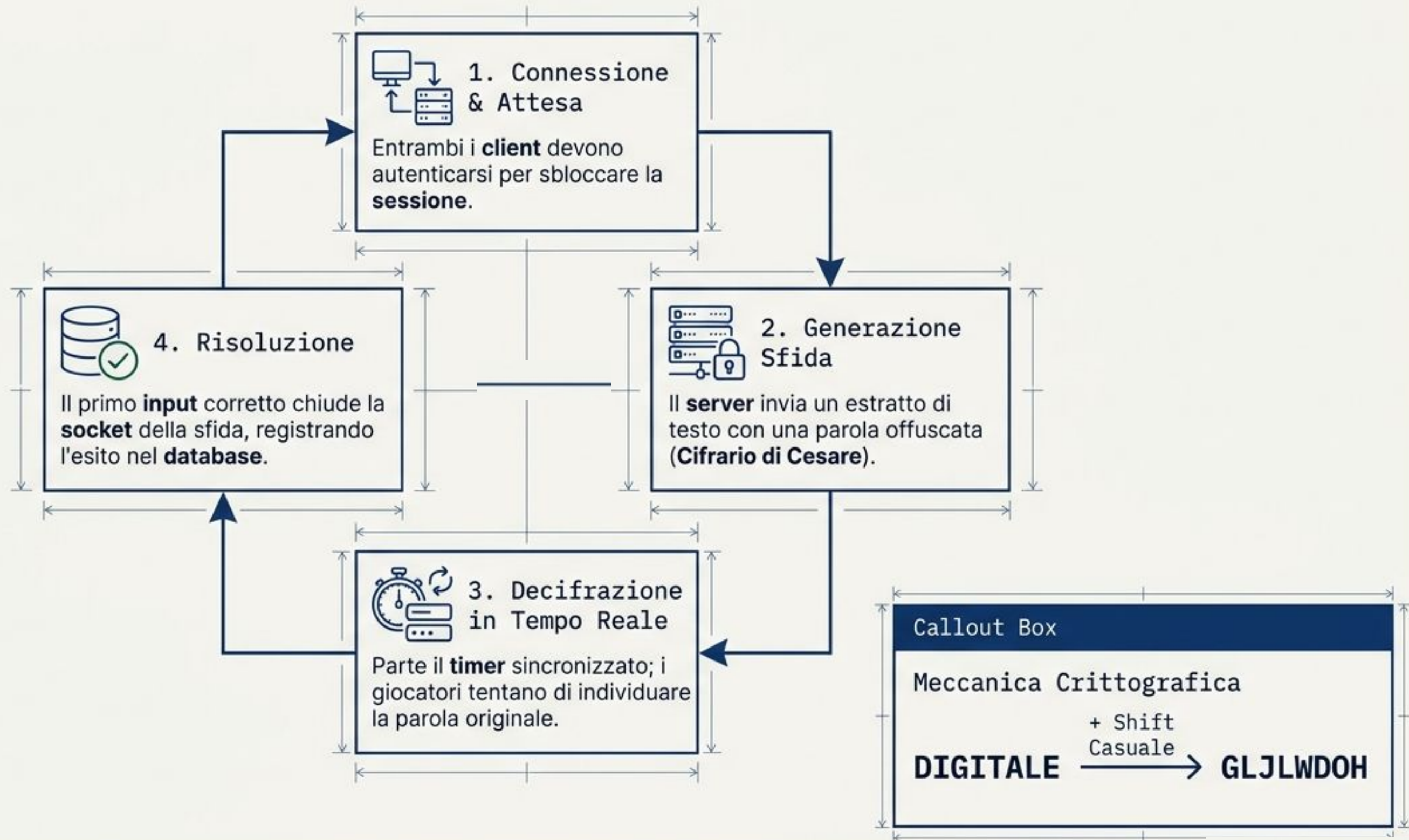
GuessTheWord — IndovinaLaParola

Progetto Finale Java A.A. 2025/2026 | Dipartimento DIEM

Un videogioco crittografico client-server in tempo reale. Due giocatori, una parola cifrata, un'unica rete.



Il Loop di Gioco Principale



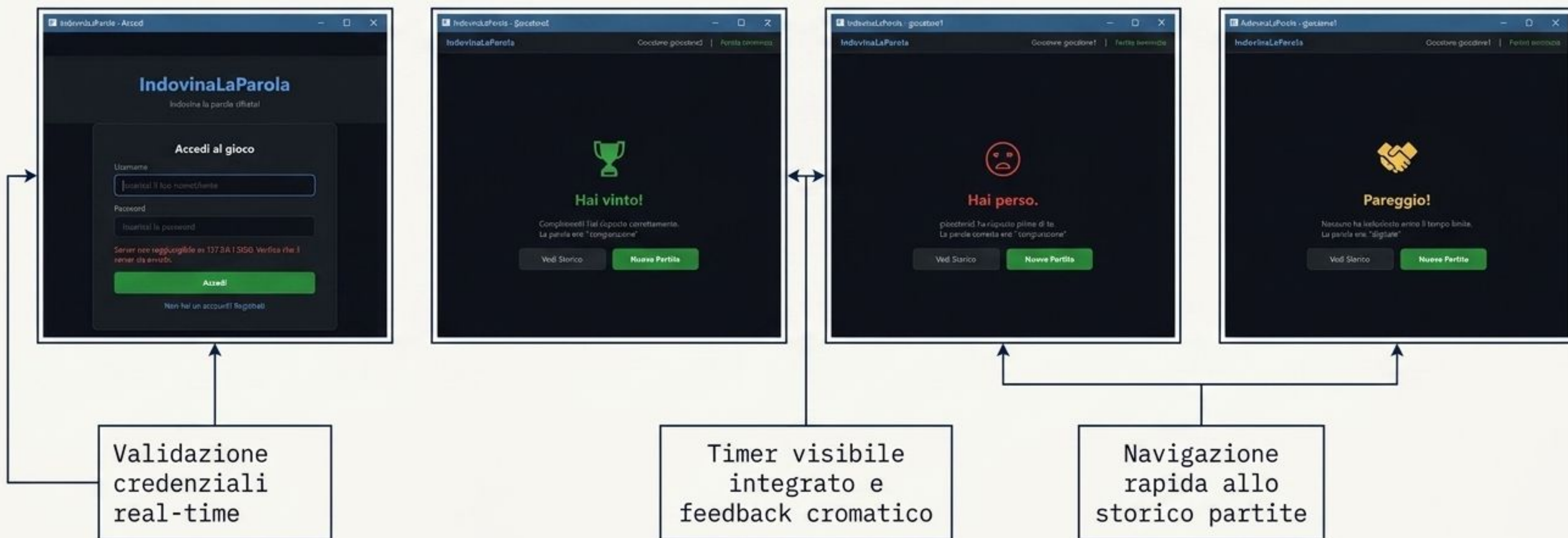
L'Esperienza del Giocatore (Interfaccia Client)

Window 1: Login Interface

Window 2: Victory Interface

Window 3: Defeat Interface

Window 4: Draw Interface



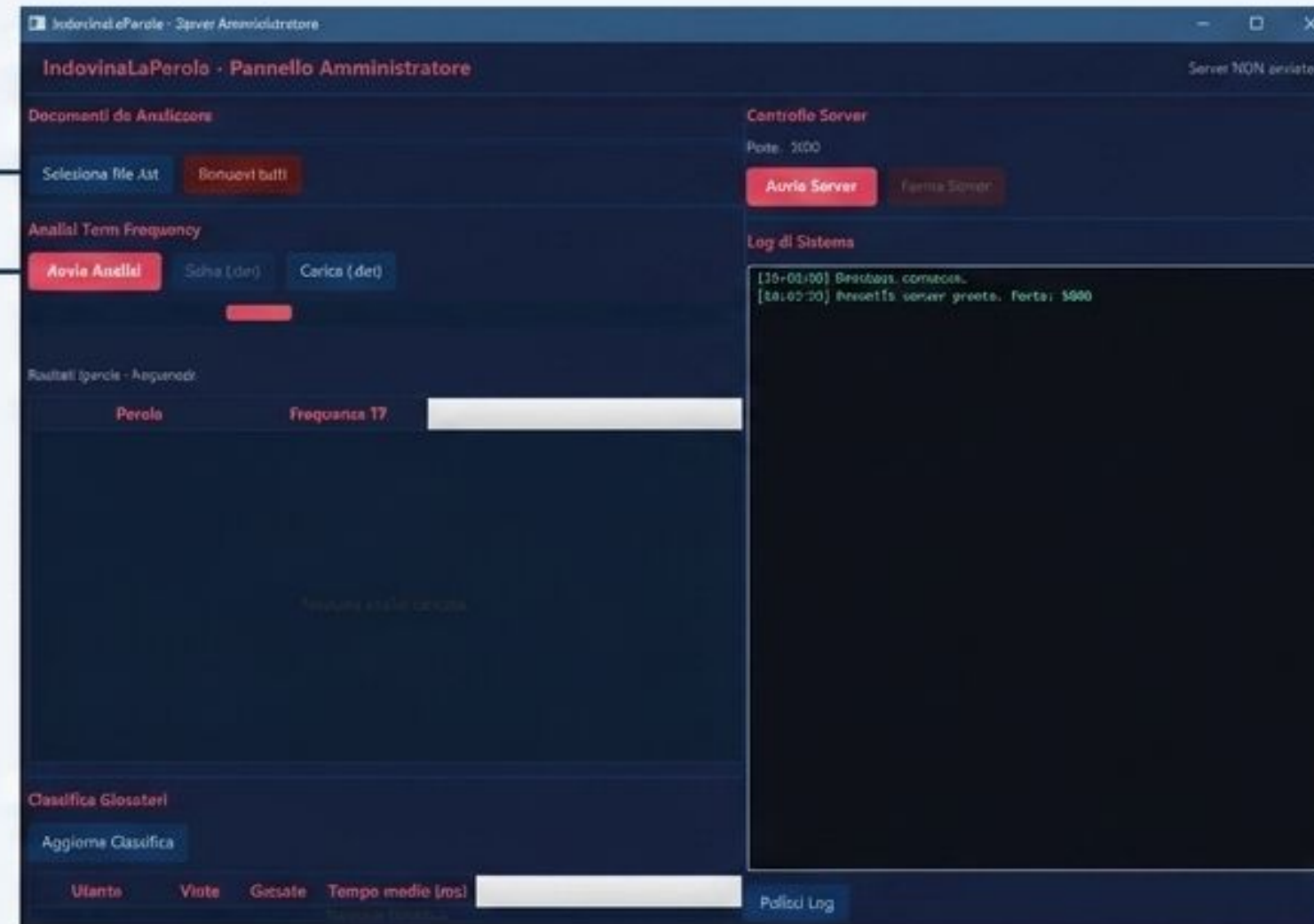
Il Pannello di Controllo (Interfaccia Amministratore Server)

Gestione Documenti

Selezione e rimozione
massiva di file .txt.

Analisi Term Frequency

Calcolo asincrono delle
frequenze.
Pulsanti per
serializzare (.dat) ed
evitare ri-elaborazioni.



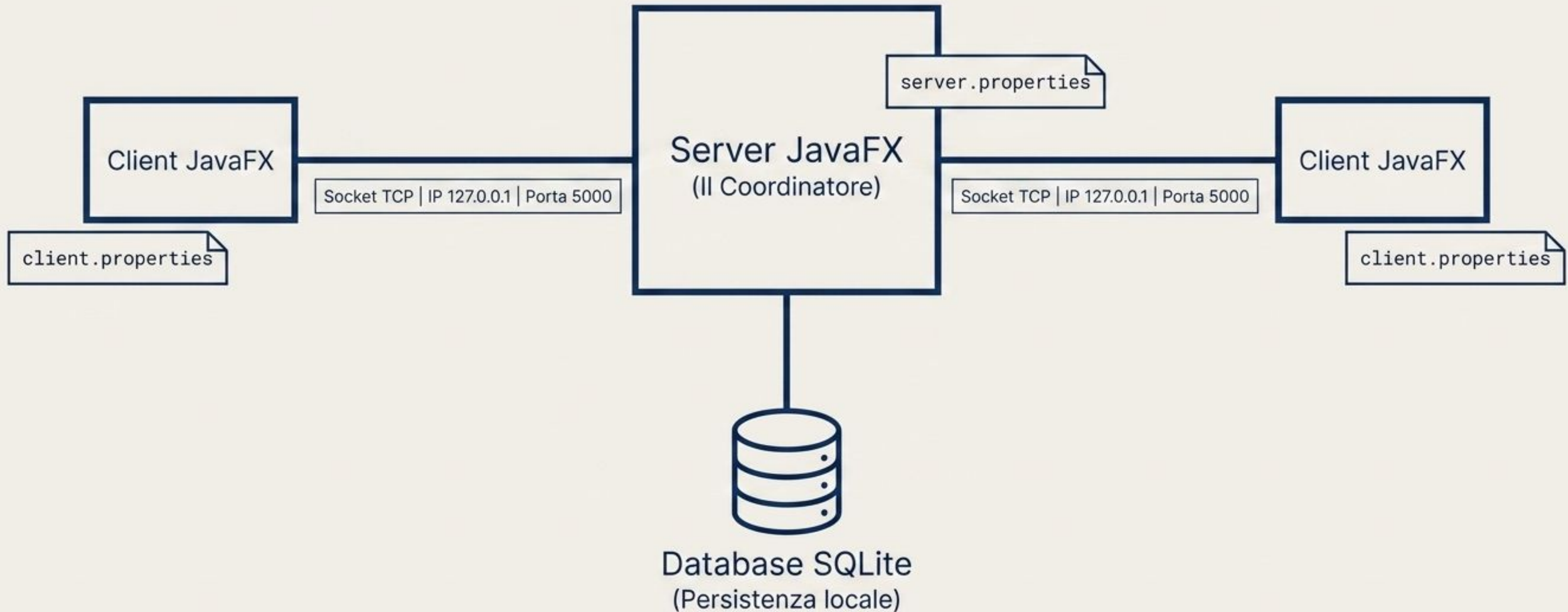
Controllo di Rete

Avvio/Arresto del
ServerSocket sulla
porta configurata.

Log Live

Monitoraggio degli
eventi di sistema (es.
connessione DB, porta
in ascolto).

Architettura di Rete Client-Server



Separazione delle Responsabilità: Server vs. Client

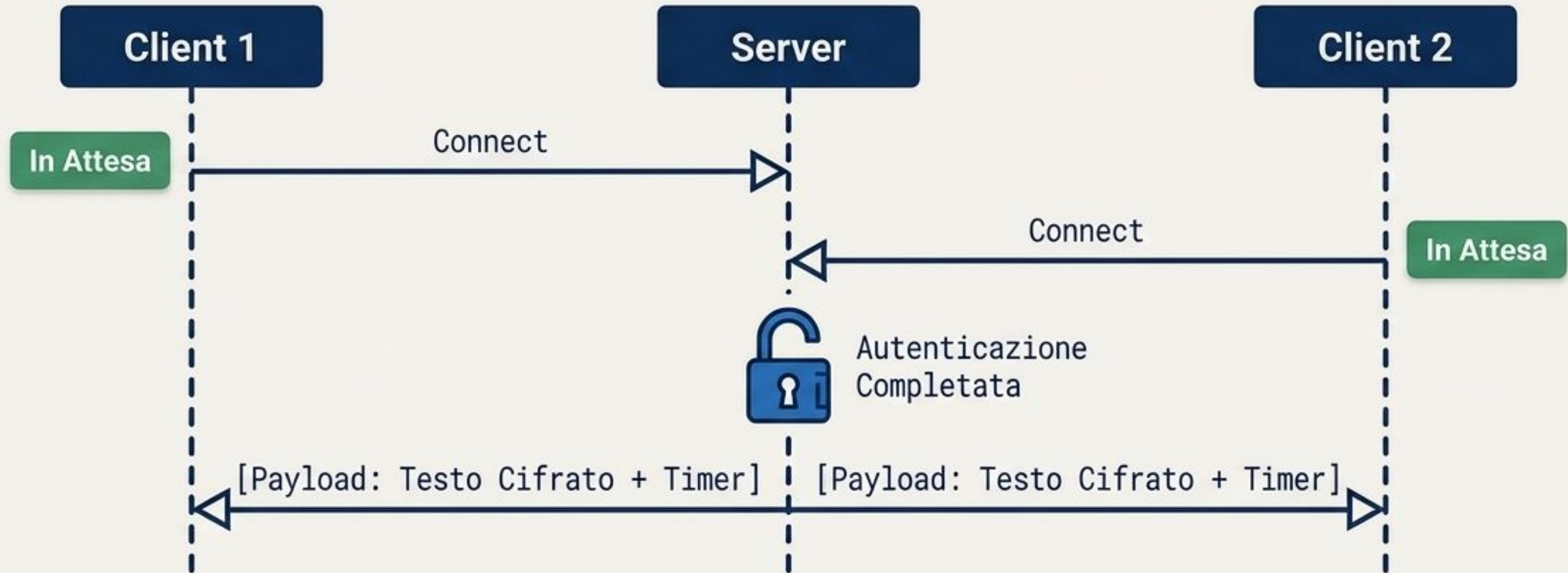
Server (Motore Logico)	Client (Rendering Visivo)
Gestione concorrente delle connessioni client.	Raccolta input utente.
Validazione delle credenziali (Login/Registrazione).	Visualizzazione sicura del testo cifrato.
Elaborazione documenti (Term Frequency) e preparazione crittografica della sfida.	Gestione grafica del timer e degli stati della partita (in attesa, in corso, terminata).
Arbitraggio della partita e validazione risposta vincente.	Richiesta e visualizzazione dello storico partite.
Scrittura su database e aggiornamento classifiche.	

Il "Challenge Engine": Pipeline di Generazione della Partita



Il processo genera stringhe univoche in tempo reale, garantendo che nessuna sfida sia identica alla precedente.

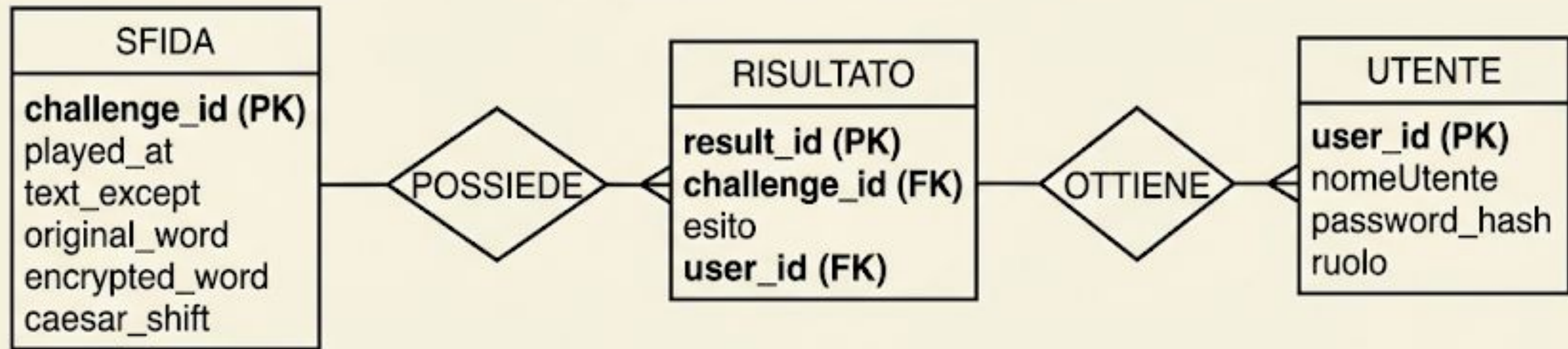
Sincronizzazione Socket e Gestione Sfida



La logica di avvio è strettamente vincolante: il timer e l'esposizione del testo avvengono esclusivamente quando due giocatori sono connessi, autenticati e pronti.

Modello Dati Relazionale (SQLite)

Schema Entità-Relazione Formale



La persistenza locale su SQLite garantisce che lo storico delle Sfide, i profili Utente e tutti i Risultati sopravvivano ai riavvii del server.

Stack Tecnologico & Vincoli di Progetto



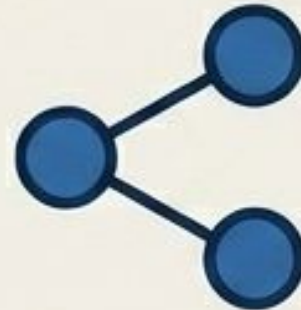
Java 8

(Compatibilità rigorosa)



JavaFX + FXML

(UI Pattern)



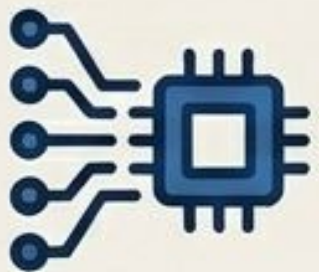
Sockets

(TCP/IP Comm)



Service & Task

(Async UI)



Stream API

(Elaborazione testo)



Serializzazione

(Salvataggio .dat)



SQLite + JDBC

(Database query)



File .properties

(Iniezione dipendenze)

Deployment e Architettura di Consegna

eseguibili.zip

eseguibili/

- server.jar (Start: java -jar server.jar)
- client.jar (Start: java -jar client.jar)
- readme.txt (Credenziali di testing incluse)
- properties/
 - server.properties
 - client.properties
- db/ (Percorsi relativi garantiti)
- documents/

Zero configurazione richiesta. Gli eseguibili includono tutti i path relativi e gli account pre-popolati (1 Admin, 2 Giocatori) per un testing di valutazione immediato.